Subsistema Controlador

1° Diogo Freitas Machado
Engenharia de Computação
Instituto de Educação Superior de Brasília - IESB
Brasília, DF
diogo.machado@iesb.edu.br
2312082012

2° Elisa Nunes de Freitas
Engenharia de Computação
Instituto de Educação Superior de Brasília - IESB
Brasília, DF
elisa.freitas@iesb.edu.br
2312082040

Orientador: Paulo Henrique Moreira de Carvalho Faria
Engenharia Eletrônica & Engenharia de Software
Instituto de Educação Superior de Brasília - IESB
Brasília, DF
paulo.faria@iesb.edu.br
201836

***Abstract*—Este documento é uma formalização das pesquisas realizadas sobre tudo o que tange a respeito do ESP32 (Controlador) para o ASV a ser desenvolvido.**

***Index Terms*—Controle, Atuadores, ESP32, motores, RIPEAM**

I. Introdução e Escopo

Uma vez estruturado o Subsistema de Alimentação, o próximo passo definido para o desenvolvimento do ASV foi a pesquisa e estruturação dos subsistemas de processamento do dispositivo. O presente documento abordará especificamente sobre o **Subsistema Controlador** e tudo o que tange a seu respeito. Dessa forma, para este documento, mencionar subsistema controlador entende-se por todas as responsabilidades , atuações e conexões do ESP32 escolhido para ser o responsável por controlar os atuadores do ASV, como motores e LEDs de segurança.

II. Hardware do Controlador (ESP32)

Antes de aprofundar nas responsabilidade, conexões, etc é necessário conhecer o microcontrolador selecionado e suas características. Sendo assim, o ESP32 foi escolhido com base nos critérios de custo, acessibilidade, facilidade de implementação/integração e capacitação para os requisitos do projeto.

A. ESP32-WROOM-32

É o microcontrolador que suprirá todos os critérios do projeto planejado visto que ele possui um custo baixo, é um dos microcontroladores mais difundidos na comunidade e é um módulo poderoso e genérico de MCU com Wi-Fi, Bluetooth e Bluetooth LE além do núcleo deste módulo com o chip ESP32-D0WDQ6, que possui dois processadores Xtensa® de 32 bits LX6 os quais podem operar a uma frequência de até 240 MHz.

Principais Componentes

- **Processador e Memória:** O módulo conta com microprocessadores dual-core Xtensa® 32-bit LX6, 448 KB de ROM, 520 KB de SRAM e 8 KB de SRAM no RTC. Além disso, possui 4 MB de memória flash SPI externa integrada.
- **Periféricos Integrados:** O módulo dispõe de uma vasta gama de periféricos, incluindo:
  - Até 32 GPIOs.
  - Interfaces como UART, SPI, SDIO, I2C, I2S e LED PWM.
  - Sensores como sensor de toque capacitivo, sensor Hall, ADC (Conversor Analógico-Digital) e DAC (Conversor Digital-Analógico).
  - Contador de pulsos e controlador TWAI® (compatível com ISO 11898-1, especificação CAN 2.0).
  - Interface Ethernet MAC para comunicações LAN.
- **Relógio e Cristal:** Possui um oscilador de cristal de 40 MHz integrado.

Por fim, este ESP possui algumas funcionalidades de destaque como Gerenciamento de Energia com diferentes modos de energia, Segurança e Boot com pinos de strapping e bits eFuse além de possuir uma gama variada de aplicações.E de todos estes componentes e funcionalidades, os mais utilizados neste sistema serão o periférico MCPWM, o periférico LEDC, as interfaces UART, a interface I2C, os GPIOs e principalmente os núcleos processadores Xtensa®.

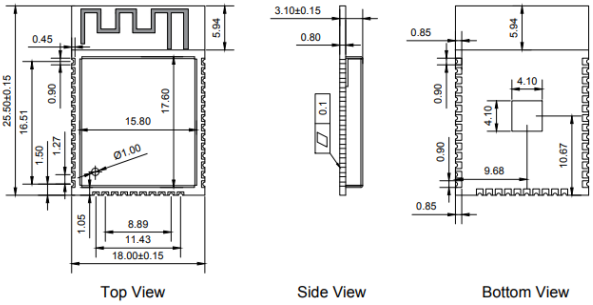

Fig. 1: Dimensões do ESP32-WROOM-32

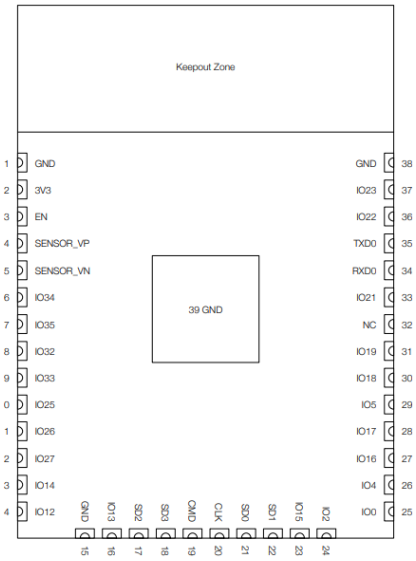

Fig. 2: Layout de Pinos do ESP32-WROOM-32

Observação.: Pretende-se utilizar o ESP32 já no modelo DEV KIT em uma placa PCB para maior simplicidade do projeto.

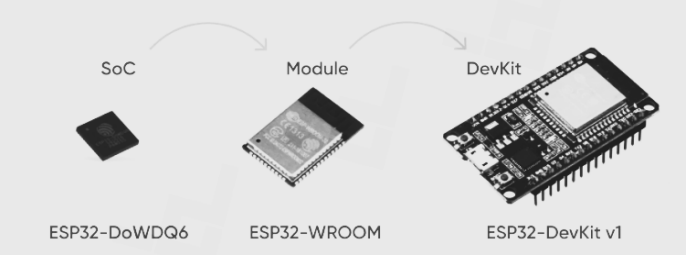

Fig. 3: Explicação visual do DEV KIT

B. Restrições de Hardware e Seleção Segura de GPIOs

Embora o ESP32 ofereça uma ampla gama de pinos de entrada e saída (GPIOs), a seleção para o controle de atuadores críticos (como motores e luzes RIPEAM) não pode ser arbitrária. Existem grupos de pinos com funções especiais de inicialização (*boot*) e restrições elétricas que, se ignoradas, podem impedir a inicialização do sistema ou causar falhas intermitentes.

Pinos de Strapping (Strapping Pins): O ESP32 possui 5 pinos de *strapping*: **GPIO 0, 2, 5, 12 e 15**. Durante a inicialização (fase de *Power-On Reset*), o chip amostra o nível de tensão nestes pinos para decidir parâmetros críticos:

- **Modo de Boot:** O GPIO 0 deve estar em nível ALTO para iniciar o firmware normalmente. Se estiver BAIXO, o chip entra em modo de gravação (*Download Mode*).
- **Tensão da Flash (VDD_SDIO):** O GPIO 12 controla a tensão do regulador interno (1.8V ou 3.3V). Se um circuito externo puxar este pino para o nível errado durante o boot, o processador pode sofrer *Brownout* e não iniciar.
- **Logs de Debug:** O GPIO 15 controla as mensagens de log na serial durante o boot.

Devido a essas características, o projeto do subsistema de controle evitou conectar *drivers* de baixa impedância ou resistores de *pull-up/pull-down* fortes nestes pinos, priorizando o uso de GPIOs "seguros" (como 16, 17, 18, 19 e 21 a 23) para o controle dos motores e comunicação.

Pinos de Entrada Exclusiva (Input-Only): Os pinos **GPIO 34, 35, 36 (VP) e 39 (VN)** não possuem circuitos de saída push-pull, funcionando apenas como entradas digitais ou analógicas. No projeto, estes pinos foram reservados estritamente para leitura de sensores (como monitoramento de tensão da bateria via ADC), evitando seu uso para controle de LEDs ou ativação de MOSFETs.

Pinos Reservados à Flash SPI: Os pinos **GPIO 6 ao 11** estão conectados internamente à memória Flash integrada do módulo ESP32-WROOM-32. O uso destes pinos para qualquer periférico externo causaria conflito no barramento SPI, travando o sistema imediatamente.

TABLE I: Mapeamento de Segurança dos GPIOs Selecionados

Grupo	GPIOs	Decisão de Projeto
Strapping	0, 2, 5, 12, 15	Evitados para saídas críticas (PWM/Motores) para garantir boot seguro.
Flash SPI	6 – 11	**Não utilizados** (Conectados à memória flash interna).
Input Only	34 – 39	Reservados estritamente para Sensores (ADC/Feedback).
Safe I/O	16, 17, 18, 19	Alocados para periféricos ativos (UART, PWM e RIPEAM).

III. Alimentação e Proteção Elétrica

Através dos reguladores Buck do Subsistema de Alimentação, será feita a conexão ao pino de alimentação (V_{in}) OU diretamente ao pino de 3V3 que permitirá a ativação e funcionamento constanste do ESP32.

IV. Arquitetura de Software (RTOS)

Diferentemente de arquiteturas de *super-loop* (execução sequencial única) comuns em microcontroladores de 8 bits, o subsistema de controle do ASV exige a execução concorrente de tarefas com requisitos temporais distintos: a malha de controle PID deve ser estritamente determinística, enquanto a interpretação de comandos via UART é orientada a eventos e assíncrona.

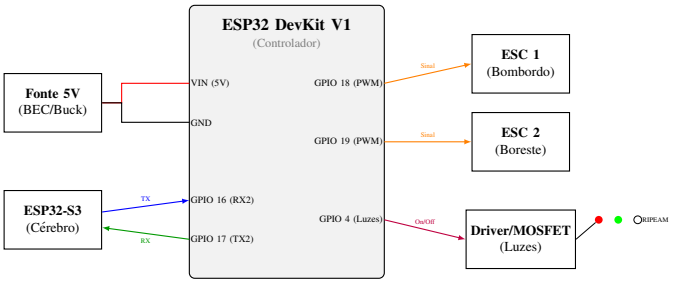

Fig. 4: Diagrama de Conexões do Subsistema de Controle (DevKit).

Para gerenciar essa complexidade, utiliza-se o **FreeRTOS** (Real-Time Operating System), nativo do *framework* de desenvolvimento do ESP32. O sistema operacional gerencia o escalonamento das tarefas (*Tasks*), permitindo preempção baseada em prioridades e o uso eficiente dos dois núcleos de processamento do ESP32-D0WDQ6.

A. Estratégia de Multiprocessamento Assimétrico (AMP)

O ESP32 possui dois núcleos Xtensa® LX6 de 32 bits. Para maximizar a estabilidade da navegação, adotou-se uma estratégia de segregação de responsabilidades:

- **Core 0 (Protocol CPU - PRO_CPU):** Dedicado às tarefas de "infraestrutura" e comunicação. Ele gerencia a pilha Wi-Fi/Bluetooth (se ativa), o monitoramento do sistema (*Watchdog*) e o processamento de pacotes da UART. Interrupções de comunicação são tratadas aqui para não inserir *jitter* (ruído temporal) no controle.
- **Core 1 (Application CPU - APP_CPU):** Dedicado exclusivamente à matemática de controle e atuação. A *Task* do PID roda isolada neste núcleo, garantindo que o cálculo das integrais e derivadas ocorra em intervalos precisos ($T_s = 50ms$), imune a atrasos causados pelo processamento de strings ou logs de sistema.

A Figura 5 ilustra essa distribuição de carga.

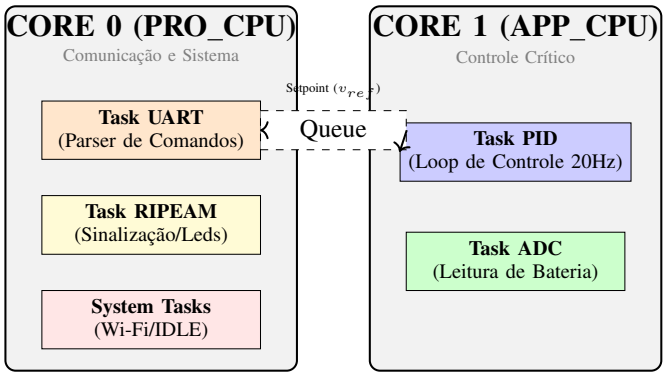

Fig. 5: Arquitetura de Software: Alocação de tarefas nos núcleos do ESP32.

B. Definição e Priorização de Tarefas

No FreeRTOS, a prioridade determina qual tarefa tem preferência de uso da CPU. O escalonador (*Scheduler*) garante que a tarefa de maior prioridade pronta para execução assuma o controle (Preempção).

A Tabela II detalha as tarefas implementadas, onde valores maiores indicam maior prioridade.

TABLE II: Mapeamento das Tasks do Sistema

Task	Core	Prio.	Freq.	Função Principal
`vTaskPID`	1	10	20 Hz	Calcular PID e atualizar PWM.
`vTaskUART`	0	7	Evento	Ler RX, decodificar JSON.
`vTaskSensors`	1	5	1 Hz	Ler tensão da bateria (ADC).
`vTaskRIPEAM`	0	3	5 Hz	Gerenciar estado dos LEDs.
`IDLE`	0/1	0	-	Gerenciamento de energia.

A `vTaskPID` possui a prioridade mais alta no Core 1 para garantir determinismo. Utiliza-se a função `vTaskDelayUntil()` para assegurar que o período de amostragem seja exato, compensando o tempo de execução do próprio algoritmo matemático.

C. Sincronização e Comunicação Inter-Processos (IPC)

Como as tarefas rodam em núcleos diferentes, elas não podem simplesmente acessar variáveis globais simultaneamente sem risco de corrupção de memória (*Race Condition*).

A troca de dados entre a recepção de comandos (Core 0) e o controle (Core 1) é realizada através de **Filas do FreeRTOS (Queues)**.

1) Quando a `vTaskUART` recebe um novo comando de velocidade, ela "posta" o valor na fila `xQueueSetpoint`.
2) No início de cada ciclo, a `vTaskPID` verifica se há novos dados na fila.
3) Se houver, atualiza a referência (SP); se não, mantém a anterior e executa o cálculo.

Este mecanismo desacopla os tempos de execução: o sistema pode receber vários comandos via Serial sem interromper a geração suave de PWM para os motores.

V. Aquisição de Dados (Sensores e ADC)

Para garantir a autonomia e a segurança do ASV, o subsistema de controle deve monitorar continuamente o estado energético da embarcação. Diferente de sistemas digitais puros, a leitura da bateria exige a conversão de grandezas analógicas (tensão contínua) para valores digitais processáveis pelo microcontrolador.

Para esta função, utiliza-se o conversor Analógico-Digital (ADC) do tipo SAR (*Successive Approximation Register*) integrado ao ESP32, configurado para operar nos pinos de entrada exclusiva (*Input-Only*), minimizando interferências de circuitos digitais.

A. Condicionamento de Sinal (Hardware)

A bateria Li-Po 4S utilizada no projeto possui tensão nominal de $14.8V$ e tensão de carga máxima de $16.8V$. Como os pinos do ESP32 toleram no máximo $3.3V$ (ou $3.6V$ absolutos), a conexão direta é destrutiva.

Implementou-se um circuito divisor de tensão resistivo para atenuar o sinal de entrada para uma faixa segura (0 a 3V),

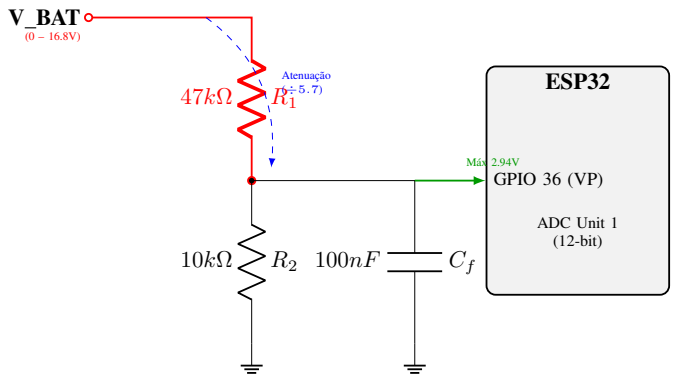

Fig. 6: Circuito de condicionamento de sinal: Divisor resistivo com filtro passa-baixa.

preservando a resolução do ADC. A topologia do circuito é ilustrada na Figura 6.

A relação de tensão é dada por:

$$V_{adc} = V_{bat} \cdot \frac{R_2}{R_1 + R_2} \tag{1}$$

Utilizando $R_1 = 47k\Omega$ e $R_2 = 10k\Omega$, obtém-se um fator de divisão de ≈ 0.175. Para $V_{bat} = 16.8V$, a tensão no pino será $2.94V$, o que está dentro da faixa linear segura do ADC com atenuação de 11dB. O capacitor C_f atua como um filtro passa-baixa passivo de primeira ordem, eliminando ruídos de alta frequência provenientes dos motores.

B. Feedback de Propulsão (Tacometria)

O controle PID de velocidade exige a realimentação da rotação real (RPM) dos propulsores. Considerando que motores *Brushless* (BLDC) de aplicação náutica frequentemente operam na configuração *Sensorless* (sem sensores Hall integrados), a estratégia de leitura baseia-se na detecção da frequência de comutação elétrica das fases.

A técnica empregada consiste no monitoramento de uma das três fases de alimentação do motor. O sinal de tensão nesta fase é uma onda trapezoidal ou senoidal cuja frequência (f_{pulso}) é proporcional à rotação mecânica, dada pela relação:

$$RPM = \frac{f_{pulso} \times 60}{N_{polos}/2} \tag{2}$$

Onde N_{polos} é o número de polos magnéticos do rotor.

Para interfacear este sinal de alta tensão (que oscila entre 0V e a tensão da bateria, ex: 14.8V) com o ESP32 (3.3V), projetou-se um circuito condicionador com acoplamento resistivo e proteção *Zener*, conforme ilustrado na Figura 7.

O divisor resistivo (R_{high}/R_{low}) reduz a amplitude da tensão da fase para níveis compatíveis com a lógica digital, enquanto o diodo Zener de 3.3V atua como *clamping*, protegendo o pino do ESP32 contra picos de tensão indutiva comuns em cargas motrizes. O sinal resultante é uma onda quadrada limpa, cuja frequência é contabilizada pela unidade PCNT do microcontrolador.

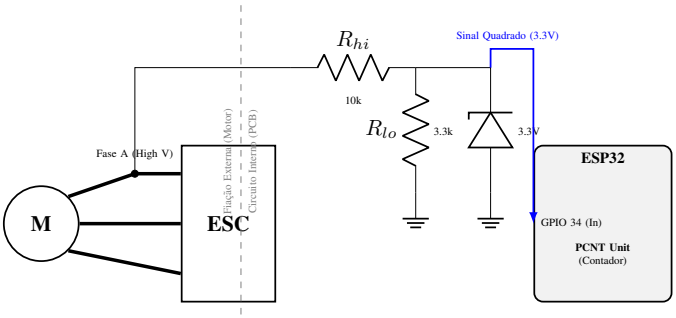

Fig. 7: Interface de leitura de RPM com proteção Zener e isolamento de domínios.

C. Sensoriamento Ambiental (LDR e RIPEAM)

O sistema de iluminação RIPEAM deve operar de forma autônoma em relação à luminosidade ambiente. Para isso, utiliza-se um sensor foto-resistivo (LDR) conectado a um pino ADC.

A simples leitura do valor de luz não é suficiente devido ao fenômeno de oscilação na linha de crepúsculo (pôr do sol), onde pequenas variações de sombra poderiam fazer as luzes ”piscarem”. Para resolver isso, implementou-se uma lógica de **Histerese de Software** baseada no diagrama da Figura 8.

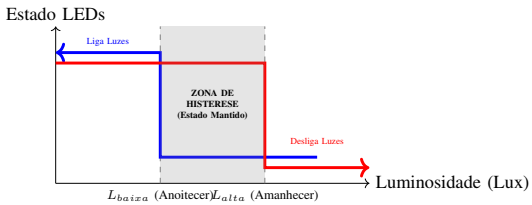

Fig. 8: Lógica de Histerese para evitar oscilação (flicker) das luzes de navegação.

O algoritmo define dois limiares distintos: um para ligar ($L_{baixa} < 300$ lux) e outro superior para desligar ($L_{alta} > 600$ lux). Enquanto a luminosidade estiver na ”Zona de Histerese”, o sistema mantém o estado anterior, garantindo estabilidade operacional.

D. Características e Calibração do ADC

O ADC do ESP32 possui resolução configurável de até 12 bits ($2^{12} = 4096$ valores discretos). No entanto, este componente é conhecido por sua não-linearidade nas extremidades da escala (próximo a 0V e 3.3V) e variação significativa do Vref (Tensão de Referência) entre diferentes chips.

Para mitigar esses erros e obter uma leitura confiável, adotou-se o seguinte fluxo de processamento digital, detalhado na Figura 9.

1) Técnicas de Software Empregadas:

- **Multisampling:** A cada ciclo de leitura, o firmware captura 64 amostras consecutivas e calcula a média aritmética. Isso reduz o ruído aleatório induzido pelos ESCs.

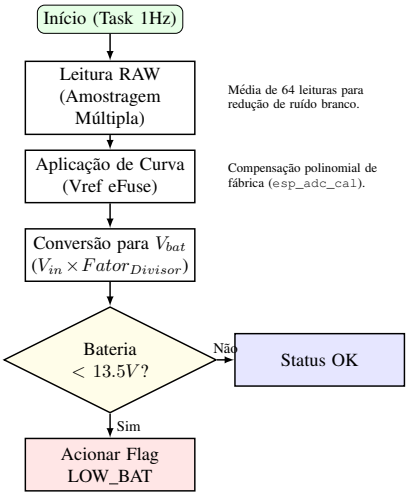

Fig. 9: Fluxo de processamento do sinal analógico e lógica de segurança.

- **Calibração por eFuse:** Utiliza-se a API `esp_adc_cal` que lê os coeficientes de calibração gravados permanentemente na memória eFuse do chip durante a fabricação, corrigindo o desvio da tensão de referência interna ($1100mV \pm 10\%$).
- **Atenuação de Hardware:** O ADC é configurado com `ADC_ATTEN_DB_11`, permitindo leituras de escala completa até aproximadamente $3.1V$, cobrindo a saída máxima do divisor de tensão ($2.94V$).

E. Diferenciação: Monitoramento de Segurança vs. Gestão Energética

É importante aqui, deixar claro a diferença entre a função da leitura de bateria realizada pelo ESP32 Controlador ("Músculo") daquela realizada pelo ESP32-S3 ("Cérebro"). Embora ambos utilizem a mesma grandeza física (tensão da bateria), seus propósitos no sistema são distintos e complementares:

- **Nível Tático (ESP32 Controlador):** Realiza a **Proteção de Hardware**. O algoritmo do ADC verificado nesta seção atua como um fusível digital de resposta rápida. Se a tensão lida cair abaixo do limiar crítico de descarga profunda ($V_{crit} = 13.2V$ para Li-Po 4S), o Controlador tem autonomia para ignorar os comandos de velocidade do Cérebro e forçar a parada dos motores (*Cut-off*), preservando a integridade química das células da bateria.
- **Nível Estratégico (ESP32-S3):** Realiza a **Otimização da Missão**. O Controlador envia o valor médio de tensão via UART para o Cérebro. O S3 utiliza este dado para recalcular a autonomia restante e ajustar o *Setpoint* de velocidade de cruzeiro, garantindo que a energia seja suficiente para completar a rota planejada via GPS.

A Figura 10 ilustra este fluxo de decisão hierárquico.

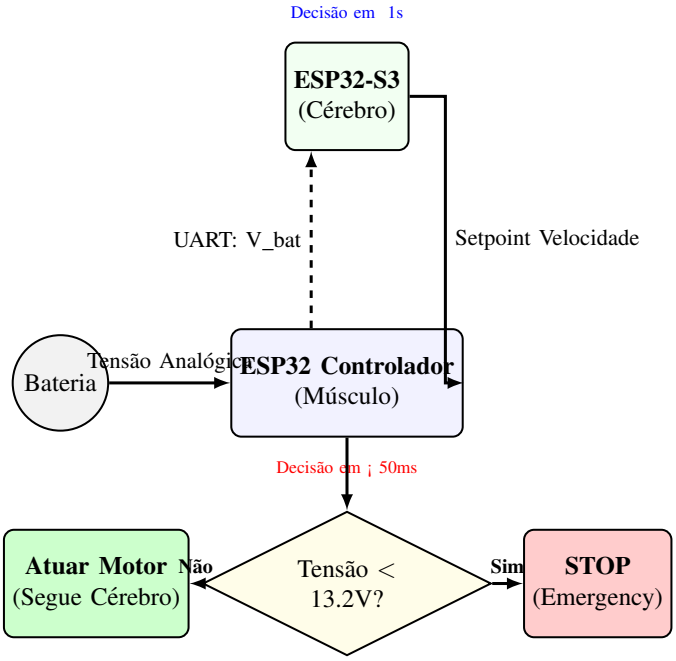

Fig. 10: Hierarquia de decisão energética: O Cérebro otimiza, o Músculo protege.

VI. Controlador PID Digital

Uma das principais responsabilidades do Subsistema Controlador será a estabilidade de navegação, essa estabilidade de navegação do ASV depende da capacidade do sistema de rejeitar distúrbios hidrodinâmicos (ondas, correntes) e manter a velocidade de referência (v_{ref}) definida pela estratégia de energia do processador principal. Para tal, implementa-se um compensador Proporcional-Integral-Derivativo (PID) em tempo discreto, executado em tempo real pelo núcleo do ESP32.

A. Discretização do Modelo Matemático

A lei de controle clássica no domínio contínuo é definida por:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{3}$$

Para implementação em microcontrolador, esta equação deve ser discretizada com um período de amostragem fixo T_s. Utilizando o método de aproximação retangular para a integral (Euler) e diferenças finitas para a derivada, obtém-se a equação de diferenças para o instante atual k:

$$u[k] = P[k] + I[k] + D[k] \tag{4}$$

Onde os termos são calculados recursivamente:

- **Termo Proporcional:** $P[k] = K_p \cdot e[k]$
- **Termo Integral:** $I[k] = I[k-1] + K_i \cdot T_s \cdot e[k]$
- **Termo Derivativo:** $D[k] = \frac{K_d}{T_s} \cdot (e[k] - e[k-1])$

O erro $e[k]$ é calculado como $e[k] = SP[k] - PV[k]$, onde SP é o *SetPoint* e PV é a Variável de Processo (velocidade medida).

B. Arquitetura de Firmware e Otimização

O algoritmo de controle é executado em uma *Task* dedicada do FreeRTOS com alta prioridade, garantindo determinismo temporal (tempo de execução previsto). O período de amostragem (T_s) é fixado em 50ms ($20Hz$), suficiente para a dinâmica lenta do veículo aquático.

Graças à arquitetura Xtensa® 32-bit LX6 do ESP32, que possui uma Unidade de Ponto Flutuante (FPU) de hardware, todos os cálculos do PID são realizados utilizando variáveis do tipo `float` sem penalidade significativa de desempenho, garantindo precisão na acumulação do termo integral e evitando erros de arredondamento comuns em aritmética de inteiros.

C. Tratamento de Saturação e Anti-Windup

Como o atuador (Driver ESC) possui limites físicos de entrada (PWM de 0% a 100%), a saída do PID deve ser limitada (*Clamping*):

$$u_{sat}[k] = \begin{cases} u_{max} & \text{se } u[k] > u_{max} \\ u_{min} & \text{se } u[k] < u_{min} \\ u[k] & \text{caso contrário} \end{cases} \tag{5}$$

Para evitar o fenômeno de *Integral Windup* — em que o termo integral cresce indefinidamente enquanto o atuador está saturado, causando sobressinal excessivo na recuperação —, implementa-se uma lógica de travamento condicional: a integração $I[k]$ é interrompida sempre que o atuador estiver saturado ($u[k] \neq u_{sat}[k]$) e o erro tiver o mesmo sinal da saturação.

D. Sinal de Controle e Atuação

O sinal de controle resultante $u_{sat}[k]$ é mapeado para o ciclo de trabalho (*Duty Cycle*) do periférico MCPWM (*Motor Control PWM*), que gera os pulsos de controle para os ESCs com resolução de microssegundos, garantindo uma resposta suave e linear dos propulsores.

E. Algoritmo de Controle Embarcado

A implementação do controlador utiliza o sistema operacional de tempo real (FreeRTOS) presente no ESP32. A lógica é encapsulada em uma tarefa de alta prioridade (`task_pid_loop`) que executa a uma frequência fixa de 20Hz.

Abaixo, apresenta-se um exemplo de trecho de código essencial que traduz a equação matemática (4) para a linguagem C++, incluindo as proteções de saturação e *anti-windup*.

```cpp
// Estrutura para armazenar estado do PID
struct PID_State {
    float Kp, Ki, Kd;
    float prev_error;
    float integral;
    float output_max;
    float output_min;
};

// Fun  o de C lculo do PID (Executada a cada ciclo)
float pid_compute(PID_State &pid, float setpoint, float measured, float dt) {
    // 1. Calcular o erro atual
    float error = setpoint - measured;

    // 2. Termo Proporcional
    float P = pid.Kp * error;

    // 3. Termo Integral (com trava condicional Anti-Windup)
    // S  integra se o sistema n o estiver saturado
    pid.integral += pid.Ki * error * dt;

    // Clamp da Integral para evitar Windup excessivo
    if (pid.integral > pid.output_max) pid.integral = pid.output_max;
    if (pid.integral < pid.output_min) pid.integral = pid.output_min;

    // 4. Termo Derivativo
    float D = pid.Kd * (error - pid.prev_error) / dt;

    // Atualiza erro anterior para o proximo ciclo
    pid.prev_error = error;

    // 5. Soma e Satura  o Final (Clamping)
    float output = P + pid.integral + D;

    if (output > pid.output_max) output = pid.output_max;
    if (output < pid.output_min) output = pid.output_min;

    return output;
}

// Task do FreeRTOS (Loop de Controle)
void task_pid_loop(void *pvParameters) {
    const TickType_t xFrequency = pdMS_TO_TICKS(50); // 50ms = 20Hz
    TickType_t xLastWakeTime = xTaskGetTickCount();

    while (1) {
        // 1. Leitura dos Sensores (Feedback)
        float velocity_now = get_speed_from_gps_or_esc();

        // 2. Leitura do Setpoint (vindo da IA/Navegacao)
        float target_speed = get_target_speed_ipc();

        // 3. Calculo do PID
        float pwm_output = pid_compute(motor_pid, target_speed, velocity_now, 0.05);

        // 4. Atuacao (MCPWM Hardware)
        set_motor_pwm(pwm_output);

        // Garante determinismo temporal (Bloqueia ate completar 50ms)
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}
```

Listing 1: Implementação da Rotina PID no ESP32

F. Cenários de Operação e Resposta Dinâmica

Para validar a robustez do algoritmo proposto, ilustram-se abaixo os dois cenários mais críticos de operação.

1) Cenário 1: Aceleração Inicial (Resposta ao Degrau): Ao receber um comando de partida ($SetPoint$ altera de 0 para $1.5m/s$), o controlador deve elevar a potência dos motores rapidamente sem causar oscilações excessivas (*Overshoot*). A Figura 11 demonstra o comportamento esperado do sinal de controle (PWM) e da velocidade do ASV.

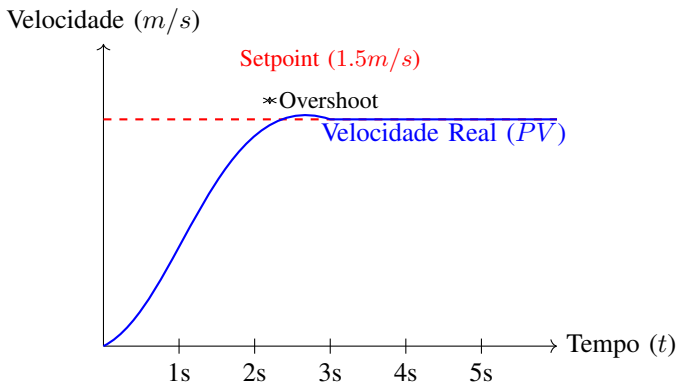

Fig. 11: Resposta teórica do PID a um comando de aceleração (Degrau).

2) Cenário 2: Rejeição de Distúrbio (Correnteza/Vento): Durante a navegação em cruzeiro, uma rajada de vento ou correnteza contrária pode frear o ASV subitamente. O termo Integral (I) do PID detecta o erro acumulado e aumenta o PWM automaticamente para compensar a força externa, como ilustrado na Figura 12.

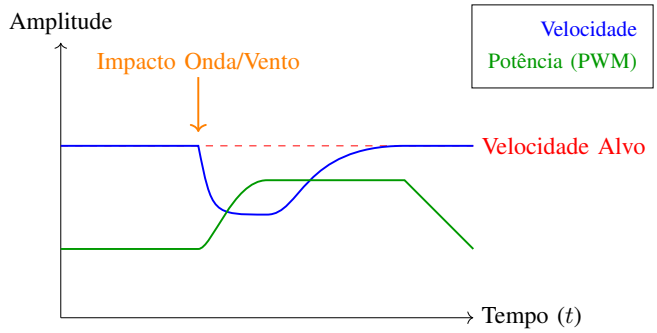

Fig. 12: Comportamento do sistema sob perturbação externa. Note o aumento do PWM (verde) para recuperar a velocidade (azul).

VII. Controle dos Atuadores (Motores e MCPWM)

O subsistema de propulsão realiza a conversão de energia elétrica em energia cinética através de motores Brushless DC (BLDC). Estes são controlados de forma independente, permitindo a navegação diferencial do catamarã e eliminando a necessidade de um leme físico.

A. Dinâmica de Propulsão Diferencial (Catamarã)

O catamarã utiliza propulsão diferencial em substituição ao leme convencional. A modelagem do sistema é dividida em dois domínios complementares:

- **Cinemática Direta:** Define como as forças resultam em movimento, considerando a inércia e a massa do veículo. Esta modelagem é a base para a previsão da resposta do ASV:

$$v_{linear} = \frac{T_{boreste} + T_{bombordo}}{m} \tag{6}$$

$$\omega_{angular} = \frac{L \cdot (T_{boreste} - T_{bombordo})}{I_z} \tag{7}$$

  Onde T representa o empuxo gerado, m a massa total e I_z o momento de inércia.
- **Cinemática Inversa:** Equação executada em tempo real pelo ESP32 para transformar o vetor de navegação (velocidade v e giro ω) em comandos individuais para cada motor:

$$V_R = v + \left(\frac{\omega \cdot L}{2}\right) \tag{8}$$

$$V_L = v - \left(\frac{\omega \cdot L}{2}\right) \tag{9}$$

  Onde L é a distância entre os centros dos cascos (*track width*).

1) Interpretação Prática e Mapeamento: A lógica de controle permite três estados fundamentais:

- **Linha Reta:** Com $\omega = 0$, ambos os motores operam em velocidades idênticas ($V_R = V_L = v$).
- **Giro sobre o Eixo:** Com $v = 0$, os motores operam com intensidades iguais e sentidos opostos ($V_R = -V_L$).
- **Curva em Movimento:** Com $v > 0$ e $\omega > 0$, o motor de Boreste (V_R) acelera em relação ao de Bombordo (V_L), por exemplo.

Após o cálculo de V_L e V_R, o sistema realiza o mapeamento (*map*) destes valores para o ciclo de trabalho (*Duty Cycle*) do periférico MCPWM. Uma velocidade de $1.5\,\mathrm{m/s}$, por exemplo, é convertida em um sinal PWM correspondente a 75% da potência do ESC.

B. Modelo do Motor Brushless (BLDC)

O motor BLDC é modelado como um sistema de segunda ordem. A determinação das constantes elétricas e mecânicas é vital para o ajuste dos parâmetros do PID (K_p, K_i, K_d), prevenindo oscilações:

$$G(s) = \frac{K}{(Ls + R)(Js + b) + K^2} \tag{10}$$

Onde R e L são a resistência e indutância das fases, e J a inércia do conjunto rotor-hélice.

C. Funcionamento do ESC (Electronic Speed Controller)

Devido à ausência de escovas físicas para a comutação de fases nos motores BLDC, o sistema utiliza ESCs que atuam como inversores trifásicos inteligentes:

- **Conversão de Sinal:** O dispositivo interpreta o sinal PWM (*Pulse Width Modulation*) de 3.3V originado no ESP32.
- **Comutação Eletrônica:** Através de uma ponte de MOSFETs, o ESC alterna a corrente entre as três fases do

motor (A, B e C), gerando o campo magnético rotativo necessário para o movimento do rotor.

- **Alimentação:** Os controladores são alimentados diretamente por uma bateria Li-Po 4S (14.8V), garantindo o suprimento para as correntes de pico exigidas em acelerações.

D. Critérios de Seleção e Especificações

Para garantir a eficiência e manobrabilidade do ASV, os ESCs selecionados atendem rigorosamente aos seguintes parâmetros:

- **Amperagem:** Dimensionados para suportar de 30A a 40A, garantindo uma margem de segurança superior a 20% em relação à corrente máxima nominal dos motores.
- **Firmware Bidirecional:** O projeto utiliza ESCs com firmware SimonK ou BLHeli configurados em modo *Bidirectional* (Forward/Reverse). Esta abordagem permite que o robô realize giros sobre o próprio eixo através da inversão de rotação de um dos motores.
- **Compatibilidade Li-Po:** Suporte nativo à tensão de 14.8V (4S).
- **BEC Interno:** O sistema utiliza o *Battery Eliminator Circuit* (BEC) integrado para regular a tensão da bateria para 5V, atuando como fonte de alimentação de redundância para os sensores.

E. Cálculos e Sinais de Controle

O controle dos ESCs estabelece-se através do protocolo padrão de servomotores, operando em uma frequência de 50 Hz (período de 20 ms). A largura de pulso (*Pulse Width*) define o estado de operação:

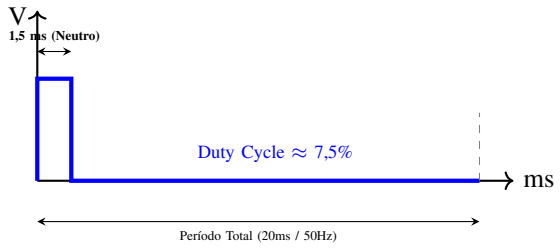

Fig. 13: Representação temporal do sinal PWM enviado aos ESCs.

- 1.0 ms (1000 µs)**:** Motor parado ou ré máxima (modo bidirecional).
- 1.5 ms (1500 µs)**:** Ponto neutro (parado no modo bidirecional).
- 2.0 ms (2000 µs)**:** Potência máxima à frente.

Para o cálculo do valor a ser carregado no periférico MCPWM do ESP32 com resolução de 16 bits ($2^{16} = 65535$), aplica-se a seguinte equação:

$$Valor_PWM = \frac{Tempo_Pulso_Desejado}{Periodo_Total} \times Resolucao \tag{11}$$

Para o estado neutro (1.5 ms):

$$Valor = \frac{1.5ms}{20ms} \times 65535 \approx 4915 \tag{12}$$

F. Implementação de Código (MCPWM do ESP32)

O controle é executado via periférico MCPWM, garantindo precisão superior ao PWM convencional.

```
#include "driver/mcpwm.h"

void configurar_motores() {
    // Inicializa o MCPWM para os dois motores
    mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0A, 18); //
    ESC 1
    mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0B, 19); //
    ESC 2

    mcpwm_config_t pwm_config;
    pwm_config.frequency = 50;    // 50Hz para ESCs
    pwm_config.cmpr_a = 0;        // 0% duty cycle
    inicial
    pwm_config.cmpr_b = 0;
    pwm_config.counter_mode = MCPWM_UP_COUNTER;
    pwm_config.duty_mode = MCPWM_DUTY_MODE_0;
    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &
    pwm_config);
}

void set_motor_speed(mcpwm_unit_t unit,
    mcpwm_generator_t gen, float speed) {
    // Mapeia -100..100 para 1ms..2ms (5% a 10% de
    duty cycle)
    float duty = 7.5 + (speed / 100.0) * 2.5;
    mcpwm_set_duty(unit, MCPWM_TIMER_0, gen, duty);
}
```

Listing 2: Configuração e Controle de Velocidade via MCPWM

G. Segurança e Ruído Eletromagnético (EMI)

Para mitigar a interferência gerada pela rápida comutação de corrente nos ESCs, as seguintes medidas são obrigatórias no projeto:

- **Capacitores de Filtro:** Inclusão de capacitores eletrolíticos na entrada de alimentação para estabilização da tensão.
- **Trançamento de Cabos:** Os cabos das fases (A, B e C) entre ESC e motor são trançados para cancelamento de campo eletromagnético.
- **Isolamento de Sinais:** As linhas de comunicação UART são posicionadas fisicamente afastadas dos cabos de potência para prevenir corrupção de dados.

VIII. Subsistema de Sinalização Náutica (RIPEAM)

A operação de Veículos de Superfície Autônomos (ASV) em corpos d'água compartilhados exige estrita conformidade com o **RIPEAM** (Regulamento Internacional para Evitar Abalroamentos no Mar). Para esta embarcação, classificada como de pequeno porte (< 7 metros) e propulsão mecânica, o sistema de sinalização visual não é apenas um acessório, mas um requisito mandatório de segurança para evitar colisões e indicar o status operacional do robô a outras embarcações.

A. Lógica de Sinalização e Estados

O controle das luzes de navegação é gerenciado pelo ESP32 (Controlador), que opera com base em duas variáveis de

decisão: *Luminosidade Ambiental* (detectada via sensor LDR ou relógio astronômico via GPS) e *Estado da Missão* (vindo da FSM).

A lógica de controle implementada segue a tabela de verdade normativa simplificada:

TABLE III: Matriz de Controle de Iluminação (Conformidade RIPEAM)

Condição Ambiental	Estado	Configuração de Luzes
Dia (> 500 Lux)	Qualquer	**Desligado** (Economia)
Noite (< 100 Lux)	Em Movimento	**Bordo** (Enc/Ver) + **Mastro** (Branca)
Noite (< 100 Lux)	Parado	**Alcançado** (Branca 360°)
Qualquer	Falha	**Estrobo** (Perigo)

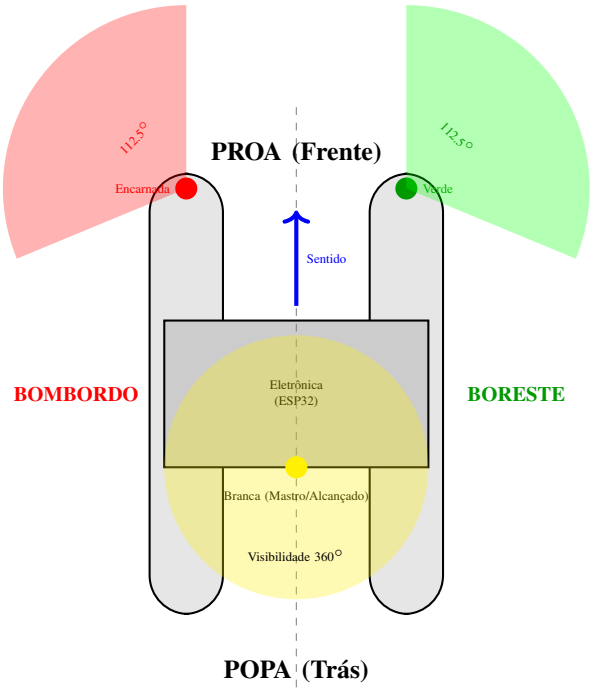

Fig. 14: Layout "Top-Down" do Catamarã: Posicionamento dos LEDs conforme RIPEAM.

B. Interface de Potência e Hardware

As luzes de navegação LED de alta luminosidade operam tipicamente com correntes na ordem de 300mA a 1A, alimentadas diretamente pela bateria de 12V (ou 14.8V).

Conforme especificado no datasheet do controlador, os pinos de I/O (GPIO) do ESP32 possuem capacidade de corrente de fonte (*source current*) limitada a 40mA. Esta corrente é insuficiente para acionar diretamente os LEDs de potência. Além disso, a tensão de operação dos LEDs (12V) excede a tensão máxima absoluta dos pinos do ESP32 (3.6V), o que causaria dano irreversível ao microcontrolador em caso de conexão direta.

Para solucionar essa incompatibilidade de tensão e corrente, projetou-se um estágio de acionamento utilizando MOSFETs de Canal N em configuração *Low-Side Switch*. O ESP32 envia um sinal lógico de 3.3V para o *Gate* do MOSFET, que então fecha o circuito de terra dos LEDs de 12V, permitindo o acionamento de alta potência com total isolamento da lógica de controle.

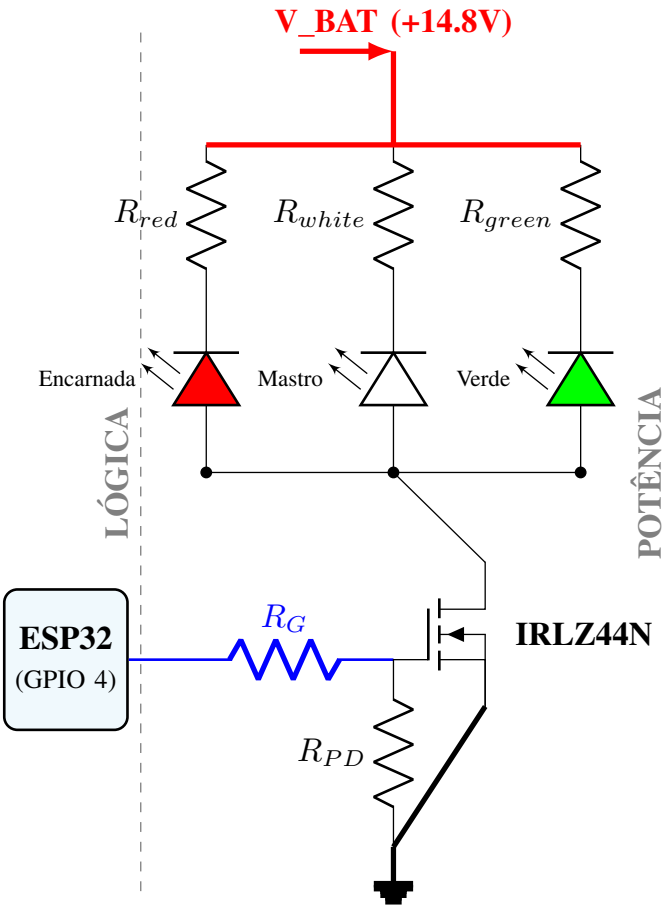

Fig. 15: Topologia de acionamento com separação clara entre estágio de potência e controle.

C. Justificativa de Segurança

A implementação deste sistema garante que, em caso de falha do software principal ("travamento"), o resistor de *pull-down* (Figura 15) garante que as luzes não fiquem piscando aleatoriamente, mantendo o estado conhecido. Adicionalmente, o uso do MOSFET permite aplicar PWM (Modulação por Largura de Pulso) caso seja necessário ajustar a intensidade luminosa para economizar bateria em situações de emergência, estendendo a autonomia de sinalização visual.

IX. Interfaces de Comunicação

A. UART (Interprocessador)

A integração entre o subsistema de processamento de alto nível (ESP32-S3 - Cérebro) e o subsistema controlador (ESP32-WROOM-32) é estabelecida via interface UART (*Universal Asynchronous Receiver-Transmitter*). Esta escolha fundamenta-se na natureza ponto-a-ponto da comunicação, garantindo um canal de dados exclusivo e direto entre os dois processadores, o que elimina conflitos de barramento e simplifica a arquitetura de rede do ASV.

O sistema utiliza o hardware dedicado (U2_UART) do ESP32 nos pinos GPIO 16 (RX2) e 17 (TX2). Esta configuração permite que periféricos internos gerenciem a transmissão de bits de forma independente, poupando ciclos de processamento da CPU principal.

1) Protocolo e Gramática de Dados (Topologia de Frame): A interface UART opera em *full-duplex*, permitindo que comandos de navegação desçam para o controlador enquanto a telemetria de sensores (velocidade, *status* da bateria e diagnóstico) sobe para o cérebro simultaneamente. Para garantir a troca de informações sem ambiguidades, o sistema utiliza uma topologia de *frame* padronizada.

A estrutura do *frame* mitiga os efeitos de interferência eletromagnética (EMI) provenientes da comutação dos motores *Brushless* através dos seguintes elementos:

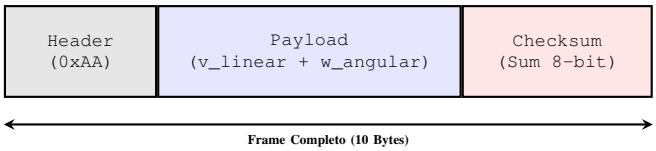

Fig. 16: Topologia do frame de comunicação UART entre Cérebro e Controlador.

- **Header (0xAA):** Sinal de atenção em hexadecimal (binário 10101010). Este padrão de onda quadrada assegura a sincronização inicial do receptor, validando o início da mensagem e descartando ruídos residuais.
- **Payload (Carga Útil):** Conteúdo real do *frame*, contendo os valores de *SetPoint* para o controle PID e comandos de acionamento de luzes.
- **Checksum (Soma de Verificação):** Selo de segurança calculado sobre o *Payload*. O controlador descarta qualquer mensagem cujo valor recebido divirja do calculado, prevenindo execuções erráticas causadas por interferência elétrica.

O firmware utiliza o *Checksum* de Soma de 8 bits (Módulo 256), garantindo alta eficiência nos núcleos de 32 bits e validação em microssegundos, mantendo o determinismo temporal do FreeRTOS. Matematicamente, a operação é definida pela Equação (13):

$$Checksum = \left(\sum_{i=1}^{n} Byte_i \right) \pmod{256} \tag{13}$$

Onde n é o total de bytes no *Payload* e $Byte_i$ representa cada valor individual. O operador módulo garante o truncamento para exatamente 1 byte.

```
// Fun  o de C lculo de Checksum (M dulo 256)
uint8_t calcular_checksum(uint8_t* payload, size_t
    tamanho) {
    uint32_t soma = 0;
    for (size_t i = 0; i < tamanho; i++) {
        soma += payload[i];
    }
    return (uint8_t)(soma & 0xFF);
}

// Valida  o na recep  o do comando
void validar_comando(uint8_t* frame_recebido) {
    uint8_t checksum_recebido = frame_recebido[
    tamanho_total - 1];
    uint8_t checksum_calculado = calcular_checksum(
    frame_recebido, tamanho_payload);

    if (checksum_calculado == checksum_recebido) {
        // Checksum v lido: comando enviado ao PID
        processar_motores();
    } else {
        // Erro detectado: descarte por EMI
        log_erro_comunicacao();
    }
}
```

Listing 3: Implementação da Validação de Checksum UART

2) Síntese da Relação de Controle e Comunicação: A integridade da mensagem via UART é a premissa para a ação física do veículo. Conforme o diagrama de conexão (Fig. 17), a tradução do dado lógico em potência ocorre sob os seguintes pilares:

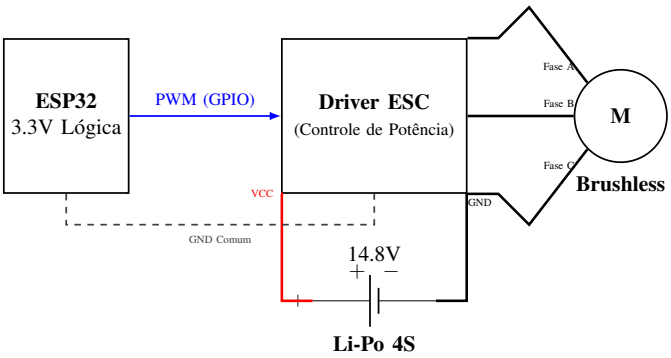

Fig. 17: Interface de Potência e Controle: Integração entre sinais lógicos e acionamento trifásico.

- **Papel dos ESCs:** O *Electronic Speed Controller* atua como estágio de potência, convertendo o sinal PWM (baseado no *Payload* da UART) em pulsos trifásicos de alta corrente para os motores, utilizando a energia da bateria Li-Po 4S.
- **Referência de GND Comum:** A conexão de GND Comum entre o ESP32 e a bateria é vital para a interpretação correta da mensagem lógica. Esta referência unificada impede a flutuação do sinal de controle e garante a estabilidade de rotação.

3) Implementação da Interpretação UART: O firmware do controlador executa uma rotina de recepção robusta para garantir que apenas mensagens legítimas sejam processadas.

```
struct ComandoNav {
    uint8_t header;      // Deve ser 0xAA
    float v_linear;      // 4 bytes
    float w_angular;     // 4 bytes
    uint8_t checksum;    // Soma de verifica  o
};

void processar_comunicacao_uart() {
    if (Serial2.available() >= sizeof(ComandoNav)) {
        if (Serial2.peek() == 0xAA) {
            ComandoNav comando;
```

```
            Serial2.readBytes((uint8_t*)&comando, sizeof(ComandoNav));

            uint8_t* payload = (uint8_t*)&comando.v_linear;
            uint8_t calc_checksum = calcular_checksum(payload, 8);

            if (calc_checksum == comando.checksum) {
                ultima_comunicacao = xTaskGetTickCount();

                float L = 0.4; // 40cm entre cascos
                float vel_boreste = comando.v_linear + (comando.w_angular * L / 2.0);
                float vel_bombordo = comando.v_linear - (comando.w_angular * L / 2.0);

                aplicar_soft_start_boreste(vel_boreste);
                aplicar_soft_start_bombordo(vel_bombordo);
            } else {
                log_erro("Checksum Invalido - Ruido EMI detectado");
            }
        } else {
            Serial2.read(); // Procura proximo header
        }
    }
}
```

Listing 4: Algoritmo de Interpretação UART e Cinemática Inversa

4) Implementação de Partida em Rampa (Soft-Start): Para proteger a integridade estrutural e evitar picos de corrente que resultariam em *resets* do microcontrolador, o sistema utiliza uma aceleração gradual.

```
float pwm_atual = 0.0;
const float TAXA_ACELERACAO = 2.0; // Incremento de 2% por ciclo

void aplicar_soft_start(float pwm_alvo) {
    if (pwm_atual < pwm_alvo) {
        pwm_atual += TAXA_ACELERACAO;
        if (pwm_atual > pwm_alvo) pwm_atual = pwm_alvo;
    } else if (pwm_atual > pwm_alvo) {
        pwm_atual -= TAXA_ACELERACAO;
    }
    set_motor_pwm(pwm_atual);
}
```

Listing 5: Logica de Soft-Start para Proteção Mecânica

5) Otimizações de Eficiência: O projeto adota estratégias para maximizar a autonomia energética:

- **Dynamic Frequency Scaling (DFS):** Redução do clock do ESP32 para 80 MHz. Esta medida economiza 30% de energia sem comprometer o controle de 20 Hz.
- **Hardware PWM (LEDC):** Utilização do periférico LEDC para geração de pulsos, garantindo estabilidade de frequência independente da CPU.

B. Watchdog de Comunicação

A segurança operacional do ASV em ambientes aquáticos é uma prioridade crítica, exigindo mecanismos que eliminem riscos de perda de controle por falhas no processador principal ou desconexão física. Para este fim, o sistema utiliza um *Watchdog* de Comunicação, que atua como um supervisor de ”batimento cardíaco” (*heartbeat*) integrado ao firmware do controlador.

1) Conceito e Diferenciação Técnica: Diferente do *Task Watchdog Timer* (TWDT) nativo do ESP32, que monitora o travamento de loops de código internos, o *Watchdog* de Comunicação supervisiona a integridade do enlace entre o Cérebro (ESP32-S3) e o Controlador (ESP32-WROOM-32). O sistema interpreta o recebimento regular de pacotes UART como o pulso vital do sistema; enquanto as mensagens são recebidas, o controlador mantém a operação ativa, validando que o comando superior está operante.

2) Lógica Matemática e Temporal: A lógica do *Watchdog* é de natureza temporal e condicional. A condição de segurança baseia-se no monitoramento do tempo decorrido desde o último pacote válido recebido (T_{ultimo_pacote}). O estado de operação normal é mantido estritamente enquanto:

$$T_{atual} - T_{ultimo_pacote} < T_{timeout} \tag{14}$$

O limite de tempo ($T_{timeout}$) é fixado em 500 ms. Este valor é definido como exatamente dez vezes o período de amostragem do PID ($T_s = 50\,\text{ms}$), estabelecendo uma margem de segurança que elimina falsos positivos. Esta configuração evita que o sistema entre em estado de falha por pequenos atrasos (*jitter*) no processamento da IA ou na latência de transmissão, garantindo a estabilidade da navegação.

3) Implementação e Failsafe (Lógica de Segurança): A implementação é realizada via software, utilizando as capacidades multitarefa do FreeRTOS, o que dispensa componentes de hardware adicionais. O algoritmo executa uma lógica de *Failsafe* rigorosa:

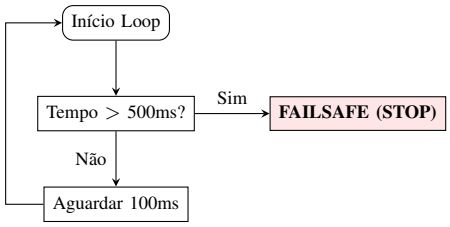

Fig. 18: Lógica de decisão do Watchdog de comunicação.

- **Reinicialização:** A cada *frame* UART validado pelo *Checksum*, o sistema zera o temporizador de segurança.
- **Estado de Emergência:** Caso a ausência de comunicação exceda os 500 ms, o controlador identifica o travamento ou desconexão do sistema de alto nível.
- **Ações Imediatas:** O subsistema interrompe instantaneamente o sinal PWM enviado aos ESCs ($PWM = 0$), cessando a propulsão para evitar colisões. Simultaneamente, o estado de missão é alterado para ”Falha”, acionando o Estrobo Vermelho (Perigo) conforme a conformidade RIPEAM detalhada na Tabela III.

Abaixo, apresenta-se a implementação da tarefa de supervisão:

```c
// Limite de seguranca definido para 500ms
const TickType_t WATCHDOG_TIMEOUT = pdMS_TO_TICKS(500);
TickType_t ultima_comunicacao = 0;

// Tarefa dedicada para monitoramento do Heartbeat
void task_watchdog_comunicacao(void *pvParameters) {
    while(1) {
        // Verifica tempo decorrido desde o ultimo pulso
        if ((xTaskGetTickCount() - ultima_comunicacao) > WATCHDOG_TIMEOUT) {
            // ENTRADA EM ESTADO DE SEGURANCA (FAILSAFE)
            set_motor_pwm(0);               // Interrompe propulsao
            acionar_sinalizacao_falha();    // Estrobo Vermelho RIPEAM
        }
        vTaskDelay(pdMS_TO_TICKS(100));     // Monitoramento a cada 100ms
    }
}

// Funcao executada na validacao de cada pacote UART
void on_valid_packet_received() {
    ultima_comunicacao = xTaskGetTickCount(); // Reinicia o Watchdog
}
```

Listing 6: Tarefa de Supervisão Heartbeat no FreeRTOS

References

[1] Kostiantyn Oliynyk, ”ESP32 Development Platforms: A Comprehensive Analysis and Comparison” (2025)

[2] Espressif, ”ESP32-WROOM-32 Datasheet Version 3.6” (2025)

[3] Norman S. Nise , ”Engenharia de Sistemas de Controle 6° Edição” (2012)

[4] Hernani B. da Cruz , Sani de C. R. da Silva e Silvio L. R. da Silva , ”Guia de Programação ESP32” (2022)

[5] L. Makridis *et al.*, “Reliable Communication Protocols for Autonomous Underwater and Surface Vehicles,” *OpenAlex*, 2023.

[6] J. Blos, Controle de motor brushless CC embarcado com partida em rampa, TCC (Engenharia Elétrica), USP São Carlos, 2018.

[7] G. C. Santos *et al.*, “Modelagem Matemática e Simulação do Motor Brushless,” *Salão do Conhecimento UNIJUÍ*, 2022.

[8] Revista Mundi, “Modelagem Matemática de um Veículo de Superfície: Dinâmica e Hidrodinâmica de um Catamarã,” *Engenharia, Tecnologia e Gestão*, vol. 8, 2023.